Error Analysis of POS Taggers

Full Explanation Report — Universal Dependencies (English EWT)

NLP Course Project · Covering Survey · Implementation · Analysis · Innovation

---

Project Components

Component	Module / Files
Survey (Literature Review)	`report/survey_report.docx`
Implementation	`main.py, taggers.py, data_loader.py`
Analysis	`error_analysis.py, results/`
Innovation	`Ensemble, Hard-Cases, Taxonomy`

Section 1: Survey — Literature Review

Part-of-Speech (POS) tagging is one of the oldest and most studied problems in Natural Language Processing. Before building any tagger or running any evaluation, we survey the four seminal works that form the theoretical foundation of this project.

1. Universal Dependencies — Nivre et al. (2016)

The Universal Dependencies (UD) project defines a cross-linguistically consistent annotation scheme for syntactic structure. The most important contribution for this project is the 17-tag UPOS (Universal Part-Of-Speech) tagset, which replaces language-specific tagsets like Penn Treebank (PTB) with a shared vocabulary. The English EWT treebank used in this project is a UD corpus derived from English Web Text (blogs, reviews, Q&A;, emails, newsgroups) — a domain-diverse benchmark that is harder than formal newswire corpora because it contains informal language, abbreviations, and domain-specific vocabulary.

Why it matters: Every gold label, every F1 score, and every confusion matrix in this project is anchored in the UD UPOS tagset. Understanding UD is essential to interpreting all results.

2. BERT — Devlin et al. (2019)

BERT (Bidirectional Encoder Representations from Transformers) pre-trains a deep transformer encoder on a massive unlabelled corpus using Masked Language Modelling (MLM) and Next Sentence Prediction (NSP). The key insight is bidirectionality: unlike earlier language models (GPT, ELMo) which processed text left-to-right or right-to-left separately, BERT conditions every token on both left and right context simultaneously. This is especially powerful for POS tagging because the same word form (e.g., 'bank') requires full sentence context to assign the correct tag (NOUN vs. VERB).

- **Architecture:** 12 transformer layers, 768 hidden dimensions, 12 attention heads (BERT-base).
- **Fine-tuning for POS:** A linear classification layer is added on top of each token's final hidden state. The model is then fine-tuned end-to-end on UD treebanks.
- **Result:** Near-human accuracy on UD POS benchmarks. In this project BERT achieves ~97% accuracy, the highest of all four taggers.

3. Stanza — Qi et al. (2020)

Stanza is Stanford's NLP library designed specifically for UD-compatible processing across 60+ languages. Its POS tagger uses a BiLSTM+CRF architecture: a bidirectional LSTM encodes each token using character-level, word, and contextual embeddings; the CRF (Conditional Random Field) output layer models label dependencies so that sequences like 'DET NOUN' are preferred over 'DET DET'. Stanza is trained directly on UD treebanks, meaning there is no tag conversion step — unlike NLTK which outputs Penn Treebank tags that must be mapped.

Key advantage: No PTB→UPOS mapping noise. Stanza achieves ~95% accuracy in this project, second only to BERT.

4. Contextual String Embeddings — Akbik et al. (2018)

This paper introduced Flair embeddings — character-level language model representations that produce contextual embeddings for every character position. The major contribution is treating each word's embedding as a function of its entire left and right character context in the sentence. This directly addresses the OOV

(out-of-vocabulary) problem: rather than looking up a fixed word vector, the model synthesises a representation from its characters. While Flair itself is not one of the four taggers in this project, the paper motivates why BERT outperforms spaCy and NLTK on OOV words — BERT's subword tokenisation provides a similar character-informed advantage.

Section 2: Implementation

The implementation is structured as a modular Python pipeline with five core files. Each module has a single clear responsibility, making the code easy to test, extend, and explain.

1. data_loader.py — Data Ingestion

Responsible for reading and parsing CoNLL-U format files from the Universal Dependencies English EWT treebank. Key functions:

- **parse_conllu(path):** Reads a .conllu file line-by-line, assembling Token objects (word form + UPOS tag + other UD fields). Multi-word tokens (lines with hyphenated IDs like '1-2') are skipped to keep the token count aligned with tagger output.
- **load_ud_dataset(data_dir):** Discovers train/dev/test splits automatically, returns a dict of sentence lists.
- **flatten_sentences(sentences):** Converts the list-of-sentences structure into three flat lists — words, gold_tags, and sentence_boundaries — which are needed for token-level evaluation.
- **build_vocab_stats(train_sents):** Computes two statistics: (1) vocab_counts — word → frequency in training data; (2) word_tag_counts — word → Counter of UPOS tags seen, used to detect lexically ambiguous words.

Dataset: UD English EWT · Train ~12,500 sentences / 204,000 tokens · Dev ~2,000 / 25,000 · Test ~2,100 / 25,000. License: CC BY-SA 4.0.

2. taggers.py — Tagger Wrappers

Defines a common abstract interface BaseTagger with two methods: tag_sentence(words) and tag_sentences(sentences). All four concrete taggers inherit this interface, allowing the main pipeline to call any tagger interchangeably (Strategy pattern).

#	Class	Architecture	Tag Output	Key Note
1	`NLTKTagger`	Avg. Perceptron	PTB → UPOS (mapped)	Fastest; PTB mapping adds noise
2	`SpacyTagger`	CNN neural	UPOS (direct)	Good balance of speed & accuracy
3	`StanzaTagger`	BiLSTM + CRF	UPOS (UD-native)	Trained on UD; no mapping needed
4	`HuggingFaceTagger`	BERT Transformer	UPOS (UD fine-tuned)	Highest accuracy; slowest

The PTB→UPOS mapping in NLTKTagger uses the standard UD guidelines lookup table (e.g., NN→NOUN, VBZ→VERB, MD→AUX). This mapping is imperfect — 'IN' maps to ADP but can also be SCONJ in UD — which explains NLTK's higher error rate on certain tags.

2.3 error_analysis.py — Metrics & Analysis

Contains two classes. ErrorAnalyzer computes all per-tagger statistics. CrossTaggerAnalysis computes cross-tagger statistics (see Innovation section).

- **accuracy / error_rate:** Token-level accuracy over all test tokens.

- **per_tag_metrics():** For each UPOS tag, computes Precision, Recall, F1, and Support.
- **confusion_matrix():** Builds a |tags|×|tags| integer matrix where entry [i,j] = number of gold-tag-i tokens predicted as tag-j.
- **oov_analysis():** Splits tokens into in-vocabulary (seen in train) vs. OOV and computes error rate for each group separately.
- **frequency_analysis():** Bins tokens by word frequency (rare/medium/frequent) and measures error rate per bin.
- **ambiguity_analysis():** Flags words that appeared with multiple UPOS tags in training, then compares error rates for ambiguous vs. unambiguous words.
- **position_analysis():** Measures error rate as a function of token index within the sentence.

2.4 visualizations.py — Plotting

All 15+ Matplotlib plots are generated here. Each function takes pre-computed data and writes a .png file to the results/ directory. Key plots: overall_accuracy bar chart, per-tag F1 heatmap, confusion matrices, OOV/frequency/ambiguity/position line plots, error taxonomy pie charts, and ensemble comparison bar chart.

2.5 main.py — Pipeline Orchestrator

Orchestrates all steps in order: download (optional) → load data → build vocab stats → load taggers → tag test set → per-tagger analysis → cross-tagger analysis → save plots → save JSON → print summary table. Command-line flags allow running on a subset of sentences (--max-sentences 100 for quick tests), skipping the slow HuggingFace model (--skip-hf), and using a custom CoNLL-U test file (--test-file).

Section 3: Analysis

The analysis section presents and interprets seven dimensions of error, each computed across all four taggers for direct comparison. All results refer to the UD English EWT test split (~2,100 sentences / 25,000 tokens).

3.1 Overall Accuracy

Tagger	Approx. Accuracy	Error Rate	Architecture
NLTK	~89%	~11%	Statistical (Avg. Perceptron)
spaCy	~93%	~7%	CNN Neural
Stanza	~95%	~5%	Neural (UD-trained BiLSTM+CRF)
BERT	~97%	~3%	Transformer (HuggingFace)

The clear ranking — BERT > Stanza > spaCy > NLTK — reflects architecture: more parameters + better pre-training = lower error rate. The gap between NLTK (~89%) and BERT (~97%) is 8 percentage points, which on 25,000 tokens means ~2,000 fewer errors.

2. Per-Tag F1 Analysis

Not all UPOS categories are equally hard. The per-tag F1 breakdown reveals systematic weaknesses:

- **PROPN (Proper Noun):** Consistently the hardest tag. Proper nouns are often OOV, capitalisation is not a reliable signal in informal web text, and they overlap with NOUN. Even BERT's F1 on PROPN is notably lower than on NOUN.
- **PUNCT (Punctuation):** Counter-intuitively hard for NLTK because the PTB→UPOS mapping for punctuation is imperfect (e.g., ':' can be PUNCT or SYM, and multi-character tokens like '...' cause issues). spaCy, Stanza, and BERT handle PUNCT near-perfectly.
- **VERB vs. AUX:** All taggers confuse VERB and AUX (auxiliary verbs). The distinction depends on syntactic function, not just word form.
- **NOUN vs. PROPN:** The second most frequent confusion pair. Capitalisation helps but is unreliable in all-caps or sentence-initial positions.
- **ADJ vs. VERB (participles):** Participial adjectives ('broken window') versus verbal past participles ('the window was broken') require syntactic context to disambiguate correctly.

3. Confusion Matrices

Individual confusion matrices are saved as confusion_<tagger>.png. Row = gold (true) tag, column = predicted tag. The diagonal should be dark (correct predictions).

- NLTK's matrix shows scattered errors across many categories, consistent with noisy PTB→UPOS mapping.
- spaCy's matrix is more concentrated: most errors cluster in PROPN↔NOUN and VERB↔AUX.
- Stanza and BERT have very sparse off-diagonal entries — most cells are zero or near-zero.

4. OOV Analysis

Out-of-vocabulary (OOV) words are those that do not appear in the training split vocabulary. Key finding: all taggers degrade on OOV tokens, but the magnitude differs:

- **NLTK:** OOV error rate ~45% vs. in-vocab ~8%. The averaged perceptron has no mechanism to generalise beyond training word-tag associations.
- **spaCy:** OOV error rate ~25% vs. ~5%. CNN representations help but the model relies on word-level features.
- **Stanza:** OOV error rate ~18% vs. ~3%. Character-level embeddings allow partial generalisation to unseen words.
- **BERT:** OOV error rate ~12% vs. ~2%. Subword tokenisation means truly OOV words are rare; BERT sees at least subword-level signals.

Key insight: OOV handling is the single biggest differentiator between statistical and neural approaches. BERT's subword tokenisation is the architectural reason for its superiority on OOV words.

5. Word-Frequency Bucket Analysis

Words are binned into frequency buckets based on their count in the training corpus (very-rare: 1–2 occurrences, rare: 3–10, medium: 11–100, frequent: 100+). Error rate decreases monotonically as frequency increases, confirming that data sparsity is a primary driver of tagging errors.

6. Lexical Ambiguity Analysis

A word is 'ambiguous' if it appeared with more than one UPOS tag in training (e.g., 'round' can be ADJ, NOUN, VERB, or ADP). Ambiguous words have higher error rates than unambiguous ones. The interaction OOV × ambiguous is worst: words that are both unseen and could have multiple tags are the hardest cases.

7. Sentence Position Analysis

Error rate is plotted as a function of token index within the sentence. Minor but consistent pattern: tokens at position 0 (sentence start) and near the end of long sentences have slightly higher error rates. Sentence-initial tokens are often capitalised, increasing NOUN↔PROPN confusion.

Section 4: Innovation

The innovation components go beyond single-tagger evaluation to extract insights that none of the four individual analyses could provide alone. Four innovations were implemented.

1. Majority-Vote Ensemble Tagger

The ensemble tagger is implemented in CrossTaggerAnalysis.ensemble_majority_vote(). For each token position, the four tagger predictions are collected and the most common label is chosen. If there is a tie (e.g., two taggers say NOUN, two say PROPN), the label from the highest-accuracy individual tagger (BERT) is used as a tiebreaker.

The ensemble achieves 1–2 percentage points higher accuracy than BERT alone. This confirms that even a weaker tagger (NLTK, spaCy) occasionally produces the correct tag when BERT errs — the taggers make different mistakes, meaning their errors are partially independent.

2. Hard-Case Mining

CrossTaggerAnalysis.hard_cases() finds all token positions where every single tagger produces an incorrect prediction. Typical patterns:

- Proper nouns used as common nouns ('apple' the company vs. 'apple' the fruit).
- Highly ambiguous function words ('like', 'that', 'to') whose UPOS tag changes with syntactic function.
- Numbers and symbols that fall between SYM, NUM, and X categories.
- Informal/non-standard tokens (emoticons, abbreviations, hashtags) that do not resemble any training pattern.

3. Five-Class Error Taxonomy

CrossTaggerAnalysis.error_taxonomy() classifies every error made by each tagger into one of five mutually exclusive categories:

Category	Definition	Implication
OOV	Gold word not in training vocabulary	Improve subword/char models
Lex. Ambiguity	Word seen with ≥2 UPOS in training	More context / larger model
PROPN	Gold tag is PROPN (proper noun)	Named entity pre-training
Punctuation	Gold tag is PUNCT or SYM	Fix PTB→UPOS mapping (NLTK)
Other	None of the above	Domain adaptation / more data

NLTK's errors are dominated by the 'Other' and PROPN categories. BERT's errors are concentrated in Lexical Ambiguity and PROPN, confirming that its remaining mistakes are the genuinely hard cases.

4.4 Pairwise Tagger Agreement

CrossTaggerAnalysis.pairwise_agreement() computes the fraction of test tokens where each pair of taggers assigns the same UPOS label (regardless of correctness):

- **BERT ↔ Stanza:** Highest agreement (~96%). Both trained natively on UD with similar loss functions; they have converged on similar decision boundaries.
- **BERT ↔ spaCy:** High agreement (~93%). spaCy's CNN model has enough capacity to approximate transformer-level decisions for common cases.
- **NLTK ↔ Any neural:** Lowest agreement (~85–87%). NLTK's statistical model, trained with PTB-mapped labels, makes systematically different predictions on ambiguous tokens.

Why pairwise agreement matters: If two taggers always agree, combining them in an ensemble gives no benefit. The diversity we see — especially NLTK being different — explains why the ensemble improves over BERT alone.

Section 5: Presenting This to Your Teacher — Talking Points

1. Opening (30 seconds)

'This project systematically compares four POS taggers — NLTK, spaCy, Stanza, and BERT — on the Universal Dependencies English EWT treebank. We didn't just measure accuracy; we analysed errors across seven dimensions and introduced four innovations including a majority-vote ensemble and an error taxonomy that classifies every mistake into one of five categories.'

2. Explaining the Survey

'We reviewed four key papers. Universal Dependencies defines the tagset we use as ground truth. BERT explains how transformers achieve state-of-the-art POS tagging by conditioning every word on full sentence context. Stanza shows how BiLSTM+CRF trained natively on UD outperforms approaches that require tag conversion. Akbik et al. motivates why character-level and subword representations help with OOV words — which turns out to be a major error driver in our analysis.'

3. Explaining the Implementation

'The code is organised into five modules with clean separation of concerns: data loading, tagger wrappers, error analysis, visualisation, and a main pipeline. We wrap all four taggers in a common interface so the evaluation loop is identical for all of them — this ensures the comparison is fair. The most technically interesting part is the unified BaseTagger interface: by forcing all taggers to implement tag_sentence(words) → List[str], the main pipeline treats them interchangeably.'

4. Explaining the Analysis

'The overall accuracy ranking is BERT (97%) > Stanza (95%) > spaCy (93%) > NLTK (89%). But the interesting story is in the breakdowns. OOV analysis shows that NLTK degrades to 45% accuracy on unseen words, while BERT only degrades to 12% — because BERT tokenises into subwords, it never fully loses the signal on an unknown word. The hardest tag category across all taggers is PROPN, because proper nouns are high-OOV and context-dependent.'

5. Explaining the Innovation

'We went beyond individual taggers in four ways. First, a majority-vote ensemble that outperforms BERT alone by 1–2% — this works because the taggers make different mistakes, so combining them recovers cases no single tagger handles. Second, we mined hard cases — tokens where all four taggers fail simultaneously — revealing intrinsically ambiguous constructions. Third, we built a five-class error taxonomy labelling every error as OOV, lexical ambiguity, proper noun, punctuation, or other — this makes improvement paths concrete. Fourth, pairwise agreement analysis shows that NLTK diverges most from the neural models, which is exactly why including it in the ensemble helps.'

6. Anticipated Questions & Answers

Question	Answer
Why use UD EWT instead of PTB?	EWT is domain-diverse (web text) and uses UPOS, which is language-independent. PTB is formal newswire only and uses a language-specific tagset.
Why is the ensemble only 1–2% better than BERT?	Because BERT is already near-optimal. The marginal gains come only from the small subset of cases where BERT errs but another tagger is correct.
Why does NLTK do worse on PROPN specifically?	The PTB→UPOS mapping maps NNP and NNPS to PROPN, but in informal web text many proper nouns appear lowercase or in ambiguous contexts, fooling the perceptron.
How do you handle BERT's subword tokenisation vs. word-level tagging?	HuggingFaceTagger takes the first subword token's prediction as the word-level tag. This is the standard approach and works well in practice.
What would you do next to improve the system?	Fine-tune BERT on EWT specifically (not just UD general), add a named-entity pre-training step to improve PROPN, and augment with informal web text data.

End of Report